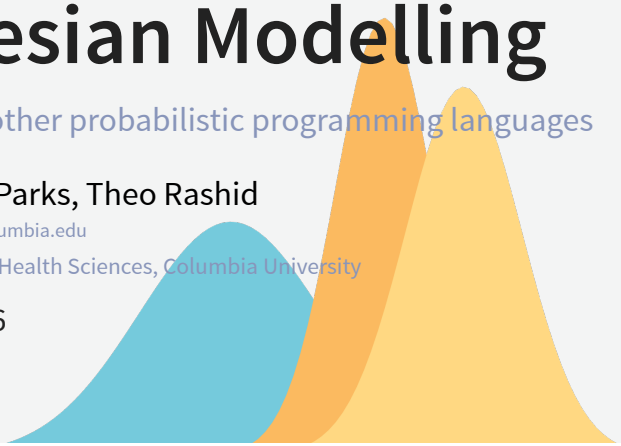
# Software Options for Bayesian Modelling

INLA and other probabilistic programming languages

Robbie M. Parks, Theo Rashid
robbie.parks@columbia.edu
Environmental Health Sciences, Columbia University

Aug 6, 2026





# What did people do before packages?



## They wrote their posterior probability density



## They wrote their posterior probability density

```python
import numpy as np
from scipy.stats import norm, uniform
from scipy.special import logsumexp

def logpdf(params, data):
    log_prior = (
      norm.logpdf(params.alpha, loc=0.0, scale=10.0) +
      uniform.logpdf(params.sigma, low=0.0, high=100.0) +
      np.sum(norm.logpdf(params.beta, loc=0.0, scale=1.0))
    )

    mu = params.alpha + jnp.matmul(x, params.beta)
    log_likelihood = norm.logpdf(data, mu, params.sigma)

    return logsumexp(log_prior + log_likelihood, axis=0)
```

## They wrote their own samplers

```python
import numpy as np

def rw_metropolis_kernel(logpdf, data, position, log_prob):
    move_proposals = np.random.normal(0, 0.1, size=position.shape)
    proposal = position + move_proposals
    proposal_log_prob = logpdf(proposal, data)

    log_uniform = np.log(np.random.uniform(low=0, high=1))
    do_accept = log_uniform < proposal_log_prob - log_prob

    position = np.where(do_accept, proposal, position)
    log_prob = np.where(do_accept, proposal_log_prob, log_prob)
    return position, log_prob
```



## We (probably) do not need to do this

- We will all (probably) make a mistake.
- Why write code that people have written before?
- Do you enjoy calibrating samplers?



# Probabilistic programming languages



# WinBUGS

# Write models in the BUGS language

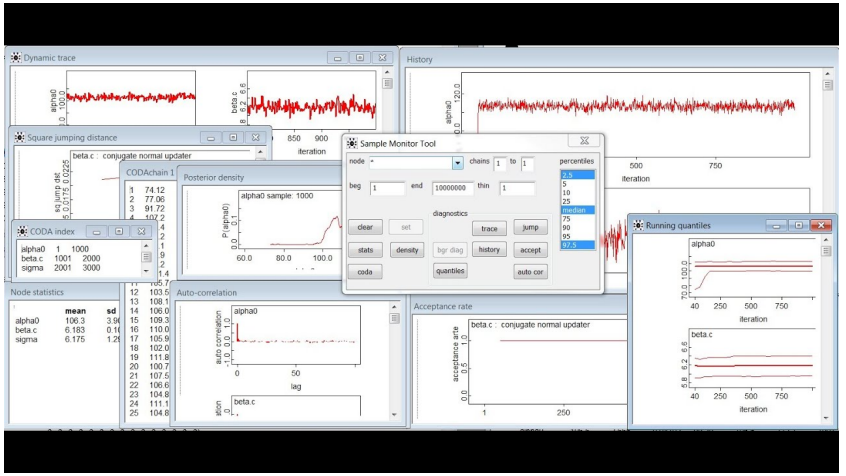
- BUGS stands for Bayesian inference Using Gibbs Sampling.
- Syntax may look familiar…

```
model(
  # prior
  alpha ~ dnorm(0, pow(10, -2))
  sigma ~ dunif(0, 100)
  tau <- pow(sigma, -2) # precision
  for (k in 1:p) beta[k] ~ dnorm(0, 1)

  # likelihood
  for (i in 1:n) {
    y[i] ~ dnorm(mu[i], tau)
    mu[i] <- alpha + inprod(beta[1:p], x[i, 1:p])
  }
)
```



# Output looks like this…






# What do we think of WinBugs nowadays?

- Point and click
- See your chains as they run
- Slow
- Old school
- Not really used widely these days…



# Stan

# Model in Stan (inspired by C++)

```
data {
  int<lower=0> N;   // number of data items
  int<lower=0> P;   // number of predictors
  matrix[N, K] x;   // predictor matrix
  vector[N] y;      // outcome vector
}
parameters {
  real alpha;           // intercept
  vector[P] beta;       // coefficients for predictors
  real<lower=0> sigma;  // error scale
}
transformed parameters {
  vector[N] mu;
  mu = alpha + X * beta;
}
```

Stan, not STAN



# The grandfather of PPLs

- The statistician's choice
- Reliable, battle-tested samplers (including NUTS)
- Interface with `python`, `R` or `Julia`
- Difficult to extend
- Writing things like `int<lower=0> N`

- Using `brms` or `rstanarm`, you can write models like base `R` (like `y ~ 1 + x`) and the inference is done in Stan.



# NIMBLE



# NIMBLE model is like WinBUGS, but more flexible.

- You are now quite familiar with `NIMBLE` syntax.

```
model(
  # prior
  alpha ~ dnorm(0, sd = 10)
  sigma ~ dunif(0, 100)
  beta[1:k] ~ dnorm(0, 1)

  # likelihood
  for (i in 1:n) {
    y[i] ~ dnorm(mu[i], sd = sigma)
    mu[i] <- alpha + (beta[1:p] %*% x[i, 1:p])[1,1]
  }
)
```

## We like NIMBLE!

- Compiles R to C++ for speed and scalability.
- Automatically finds conjugate relationships.
- Can pick a different sampler for each parameter.
- Good for teaching the fundamentals of Bayesian inference.
- If the sampler is not conjugate, it can mix poorly.

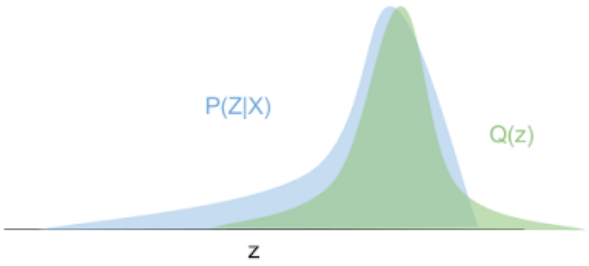


# Is sampling the only way (introducing INLA)?



## Variational inference

- Turn the inference problem into an optimisation problem.
- Minimise the difference between true posterior distribution, , and variational distribution, .






## I've heard people talk about INLA. What is INLA?

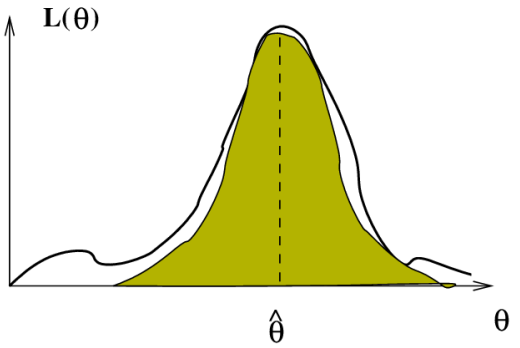
- INLA stands for Integrated Nested Laplace Approximation.
- Approximate Bayesian inference for latent Gaussian models.
- Really good for spatial statistics.
- Really quick (if you don't have too many hyperparameters).
- Most real world cases are latent Gaussian – it just works.
- A growing community of users, 2 of whom are here today.

# How does INLA work?

INLA computes accurate and fast approximations to the posterior marginals of the components of the latent Gaussian variables.



# How does INLA work?




- The Laplace Approximation (`IN'LA'`) means that we approximate the latent field with a Gaussian. Most things in the real world are Gaussian, so this works well.



# Do I fit models like all the PPLs above?

- No, actually, the syntax is quite similar to base R.
- Use R package R-INLA to fit models.
- For example, this is how to specify a model with an intercept and a random effect:

```
1 formula_iid <- y ~ 1 + f(x, model = "iid")
```

- So it's easier to specify models, and there's a lot of built in functionality.
- **But** it's more difficult to *customise*.



# Running INLA

- We will focus on INLA as it is very useful and practical, and we will use it for the remainder of the course.
- First, write the model:

```
1 formula_iid <- y ~ 1 + f(x, model = "iid")
```

- Then, run the model using the `inla` function in R:

```
1 model <- inla(
2   formula_iid,
3   data = data,
4   E = expected,
5   family = "poisson",
6   control.predictor = list(link = 1), # log-link
7   control.compute = list(config = TRUE) # so that we can simulate
8                                         # draws of posterior
9 )
```

# Customising priors in INLA

- If you don't specify priors in INLA, it will choose some default ones.
- The formula from before gives us an IID effect, but no control over the hyperprior for the variance of the effect.

```
formula_iid <- y ~ 1 + f(x, model = "iid")
```

- INLA usually chooses some good defaults for you, but here, we can put a non-default prior on the precision (1/sigma**2).

```
formula_iid_priors <- deaths ~ 1 + f(SIGLA, model = "iid",
    hyper = list(prec = list(prior = "loggamma", param = c(0.01, 0.01))))
```



# Previous examples from NIMBLE in INLA



# Linear regression

- NIMBLE formula:

```
code <- nimbleCode({
  # priors for parameters
  alpha ~ dnorm(0, sd = 100) # prior for alpha
  beta1 ~ dnorm(0, sd = 100) # prior for beta1
  beta2 ~ dnorm(0, sd = 100) # prior for beta2
  beta3 ~ dnorm(0, sd = 100) # prior for beta3
  sigma ~ dunif(0, 100) # prior for variance components

  # regression formula
  for (i in 1:n) {
    # n is the number of observations we have in the data
    mu[i] <- alpha + beta1 * x1[i] + beta2 * x2[i] + beta3 * x3[i] # manual
    y[i] ~ dnorm(mu[i], sd = sigma)
  }
})
```



# Linear regression

- INLA formula (default priors):

```
formula_linear <- y ~ 1 + x1 + x2 + x3
```

- Run model using inla function:

```
model_linear <- inla(
  formula_linear,
  data = data,
  family = "gaussian")
```

# Linear regression

- INLA formula (custom priors):

```
formula_linear <- y ~ 1 + x1 + x2 + x3
```

- Run model using inla function:

```
model_linear <- inla(
  formula_linear,
  data = data,
  family = "gaussian",
  ccontrol.fixed = list(
    mean = 0,
    prec = 0.0001
  ),
  control.family = list(
    hyper = list(
      prec = list(
        prior = "loggamma",
        param = c(1, 0.0001)
      )
    )
  )
)
```



# Hierarchical model

- NIMBLE formula:

```
partial_pooling_model <- nimbleCode({
  # priors
  alpha ~ dnorm(0, 5)
  sigma_p ~ T(dnorm(0, 1), 0, Inf) # half-normal

  for (j in 1:Np) {
    theta[j] ~ dnorm(0, sd = sigma_p)
  }

  # likelihood
  for (i in 1:N) {
    y[i] ~ dpois(mu[i])
    log(mu[i]) <- log(E[i]) + alpha + theta[province[i]]
  }
})
```



# Hierarchical model

- INLA formula (custom priors):

```
formula_partial_pooling <- y ~ 1 + f(SIGLA, model = "iid",
            hyper = list(
              prec = list(
                prior = "pc.prec",
                param = c(1, 0.01)
              )
```

- Run model using inla function:

```
model_partial_pooling <- inla(
  formula_partial_pooling,
  data = data,
  E = expected,
  family = "poisson",
  control.predictor = list(link = 1),
  control.compute = list(config = TRUE)
)
```

# Non-linear exposure-response

- NIMBLE formula:

```
Model3_ecoCode <- nimbleCode({
  # priors
  alpha ~ dflat() # vague prior (Unif(-inf, +inf))
  overallRR <- exp(alpha) # overall RR across study region

  for (j in 1:K) {
    beta[j] ~ dnorm(0, tau = 1)
  }

  # likelihood
  for (i in 1:N) {
    O[i] ~ dpois(mu[i]) # Poisson likelihood for observed counts
    log(mu[i]) <- log(E[i]) + alpha + inprod(beta[], X[i, ])
  }
})
```



# Non-linear exposure-response

- INLA formula:

```
formula_non_linear <- O ~ 1 + X1 + X2 + X3
```

- Run model using inla function:

```
model_non_linear <- inla(
  formula = formula_non_linear,
  family = "poisson",
  data = covid_data,
  offset = E,
  control.fixed = list(
    mean = c("(Intercept)" = 0, X1 = 0, X2 = 0, X3 = 0),
    prec = c("(Intercept)" = 1e-6, X1 = 1, X2 = 1, X3 = 1)
  ),
  control.compute = list(dic = TRUE, waic = TRUE, cpo = TRUE)
)
```



# Spatial model.

- You will see in lab!

## I think I'm INLA (sorry)

- Good, we'll cover more in the practical.



# Getting ready for the lab



## The lab for this session

- The goal of this lab is to use INLA to run some hierarchical models.
- During this lab session, we will:

1. Translate a model from NIMBLE into INLA;
2. Fit spatial models with INLA;
3. Learn how to work with INLA objects; and
4. See how to write models in different probabilistic programming languages.



# Questions?